

Cohortly

Software Engineering Major Project

Hamzah Ahmed

Contents

1. Definition and Analysis of Problem	1
1.1. Problem Statement and Project Proposal	1
1.2. Stakeholders	1
1.2.1. Students	1
1.2.2. Teachers	1
1.3. Specifications	1
1.3.1. Functional	1
1.3.1.1. Authentication	1
1.3.1.2. Scheduling and Study Sessions	1
1.3.1.3. Topic Hierarchy	2
1.3.1.4. Moderation	2
1.3.1.5. Q&A	2
1.3.1.6. Resource Sharing	2
1.3.1.7. Points system	2
1.3.2. Non-Functional	2
1.3.2.1. Usability	2
1.3.2.2. Security and Privacy	2
2. Research and Planning	3
2.1. Gantt Chart	3
2.2. Design	3
2.2.1. Data flow Diagram	3
2.2.1.1. Context Diagram (Level 0)	3
2.2.1.2. Level 1	4
2.2.1.3. Level 2	4
2.2.2. Structure Chart	5
2.2.2.1. Level 1	5
2.2.2.2. Level 2	5
2.2.3. Class diagram	6
2.2.4. Decision Trees	6
2.2.5. Database Schema	7
2.2.6. Data Dictionary	8
2.2.7. Algorithms	9
2.2.7.1. Main program	9
2.2.7.2. Login	10
2.2.7.3. Leaderboard	11
2.2.7.4. Questions Page	12
2.2.7.5. Session Page	12
2.2.8. Storyboard	13
2.2.9. Interface Design	13
2.2.9.1. Consistency	13
2.2.9.2. Responsive Web Design	13
2.2.9.3. Navigation	13
2.2.9.4. Grouping	13
2.2.9.5. Effective use of colour	13
2.2.9.6. Accessibility	13
3. Producing and Implementing	13
3.1. Logbook	13
3.1.1. 16 November 2025	13
3.1.2. 7 January 2026	13
3.1.3. 21 April 2026	14
3.1.4. 12 July 2026	14
3.1.5. 18 July 2026	14
3.1.6. 19 July 2026	14
3.1.7. 21 August 2026	14
3.1.8. 23 August 2026	14
3.1.9. 24 August 2026	14
3.1.10. 30 August 2026	14
3.2. User documentation	14
3.2.1. Getting Started	14
3.2.2. Joining a subject	14

3.2.3. Asking and answering questions	14
3.2.4. Uploading and using resources	14
3.2.5. Joining study sessions	14
3.2.6. Frequently Asked Questions	15
3.2.6.1. Why can't I join a session?	15
3.2.6.2. Why can't I see my subject on the dashboard?	15
3.2.6.3. How are new subject requests processed?	15
3.2.6.4. How does moderation work?	15
4. Testing and Evaluating	15
4.1. Test Report	15
4.1.1. Authentication	15
4.1.2. Sessions	15
4.1.3. Resources	16
4.1.4. Questions page	16
4.1.5. Recommendations	17
4.2. Evaluation	17
4.2.0.1. Scheduling and Study Sessions	17
4.2.0.2. Topic Hierarchy	18
4.2.0.3. Moderation	18
4.2.0.4. Q&A	18
4.2.0.5. Resource Sharing	18
4.2.0.6. Points system	18
4.2.1. Non-Functional	18
4.2.1.1. Usability	18
4.2.1.2. Security and Privacy	18

1. Definition and Analysis of Problem

1.1. Problem Statement and Project Proposal

Student cohorts perform the best when they collaborate. Stronger students in a subject can assist weaker students by sharing notes and doing peer study sessions. This also benefits the tutor, as it gives an opportunity for them to revise the material.

Currently, these sessions are uncoordinated and take place across disparate messaging applications. Study material, and answers to queries can easily get lost in these closed messaging channels, leaving out the opportunity for them to benefit others with the same questions.

A peer tutoring web-app would allow students to coordinate group study sessions, share notes and study material and have a shared set of questions and answers to post and view.

1.2. Stakeholders

1.2.1. Students

Such a system would be run most effectively when it is maintained primarily by students. Students will be both the tutors and the pupils in this system, where tutors benefit by revising the content by explaining it, and pupils benefit by catching up on the topics they may have missed in class. Additionally, some admin work is required to effectively run such a system, which should also be done by students. Student “moderators” volunteer to help keep a specific subject running smoothly by organising group sessions, and ensuring content is appropriate, correct and on-topic.

1.2.2. Teachers

A core requirement of this project is not to introduce any additional teacher workload. This system intentionally keeps teacher involvement minimal, as teacher involvement would not only create extra workload but would also affect group dynamics in a significant way. Additionally, teacher involvement would potentially create duty-of-care issues, where teachers may be liable to supervise student activity. Thus, the system does not involve teachers with any student-to-student interaction and teachers should only be able to see aggregated statistics on student progress and have access to a contribution leaderboard.

1.3. Specifications

1.3.1. Functional

Authentication

1. The system should allow teachers and students to log in using the school portal system
2. The system should allow students to:
 - Post Q&As
 - Schedule or join sessions
 - Share resources
 - Set peer tutoring availability
3. The system should allow moderators to:
 - Remove inappropriate or irrelevant content
 - Review flags for out-of-syllabus or disputed material
 - Add subtopics to subjects
4. The system should allow system administrators to:
 - Add or remove moderators
 - Create and manage subjects
5. The system should allow teachers only to:
 - View statistics on platform use (contribution credits) and student topic mastery ratings

Scheduling and Study Sessions

1. Students should be able to view public study sessions/tutoring sessions, and filter by:
 - Subject
 - Host (tutor or subject moderator)
 - Date/time
 - Sessions should show:
 - Subject or topic
 - Session title and description
 - Date and time
 - Meeting link or location
 - Host (student or moderator)
 - Participant list
 - Maximum capacity
2. Students should be able to:
 - Set availability as a peer tutor:
 - Set availability for tutoring sessions
 - Set size (max number of students) of session
 - Accept or decline requests
3. Subject moderators should be able to create general study sessions:
 - General study sessions timing is decided by scheduling polls
4. Students should be able to vote in scheduling polls:
 - Students vote for their available times, and the most popular time slot is chosen

Topic Hierarchy

1. The system should represent topics in a hierarchical way:
 - Topics are associated with a subject, and topics can have sub-topics corresponding to increasing detail
2. Students should be able to vote on the difficulty and mastery (how much they understand) of a topic
3. Teachers should be able to view aggregated statistics on difficulty and mastery metrics

Moderation

1. Students should be able to flag content as:
 - Inappropriate
 - Out of syllabus
 - Disputed
2. Moderators should be able to see a queue of flagged content, and choose an action (accept flag/delete/reject flag)
3. The system should store all moderator actions in an audit log, so moderator actions are held accountable to the moderator

Q&A

1. Students should be able to:
 - Post questions, associated with a topic in the topic hierarchy
 - Respond with answers
 - Flag a question

Resource Sharing

1. To share notes, worksheets and study materials, students should be able to:
 - Share links
 - Upload files
2. The system should tag resources with:
 - Subject
 - Topic
 - Uploader
 - Upload date
3. The system must scan uploaded files with VirusTotal API or similar
4. The system must limit to safe file formats (PDF, DOCX, PPTX, etc.)
5. Students should be able to flag resources

Points system

1. Students should be able to upvote resources, Q&A, and study sessions to indicate that they were helpful
2. After a study sessions, participants should be able to provide anonymous feedback on a study session, including:
 - A rating out of 5
 - General feedback
3. The system should provide students with **contribution credits** for participating, with higher credit amounts for more helpful activities
4. The system should display a leaderboard for the students with the highest credit amounts

1.3.2. Non-Functional

Usability

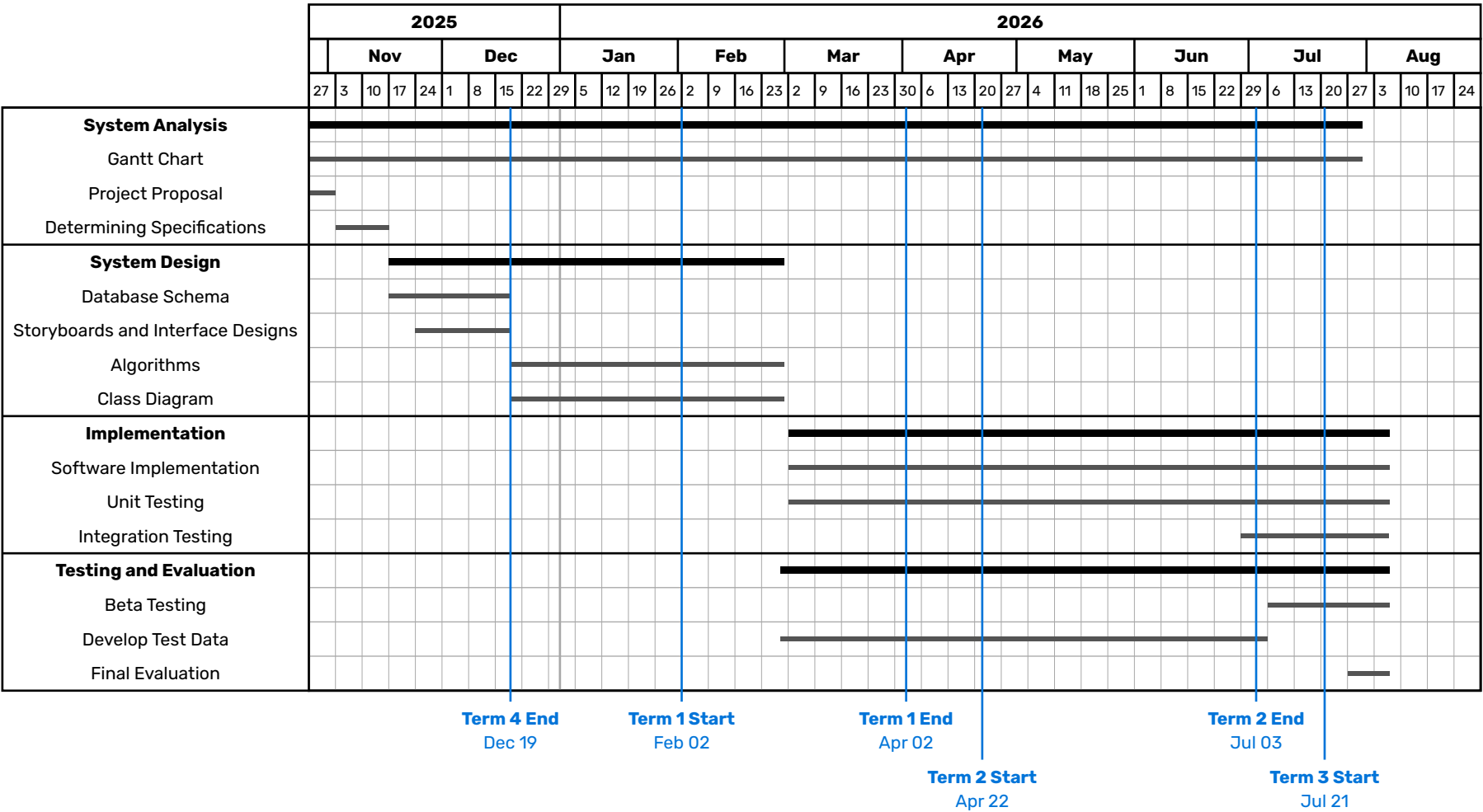
1. The system must be functional on desktop and mobile devices
2. The most commonly used actions must be easy to access intuitively
3. The interface must be consistent
4. There must be a help page to explain more complex processes (such as moderation, the contribution credits system, etc.)

Security and Privacy

1. Only students or teachers logged in through the school portal should be able to view any data within the system
2. Uploaded content should be automatically scanned with VirusTotal and restricted to safe formats

2. Research and Planning

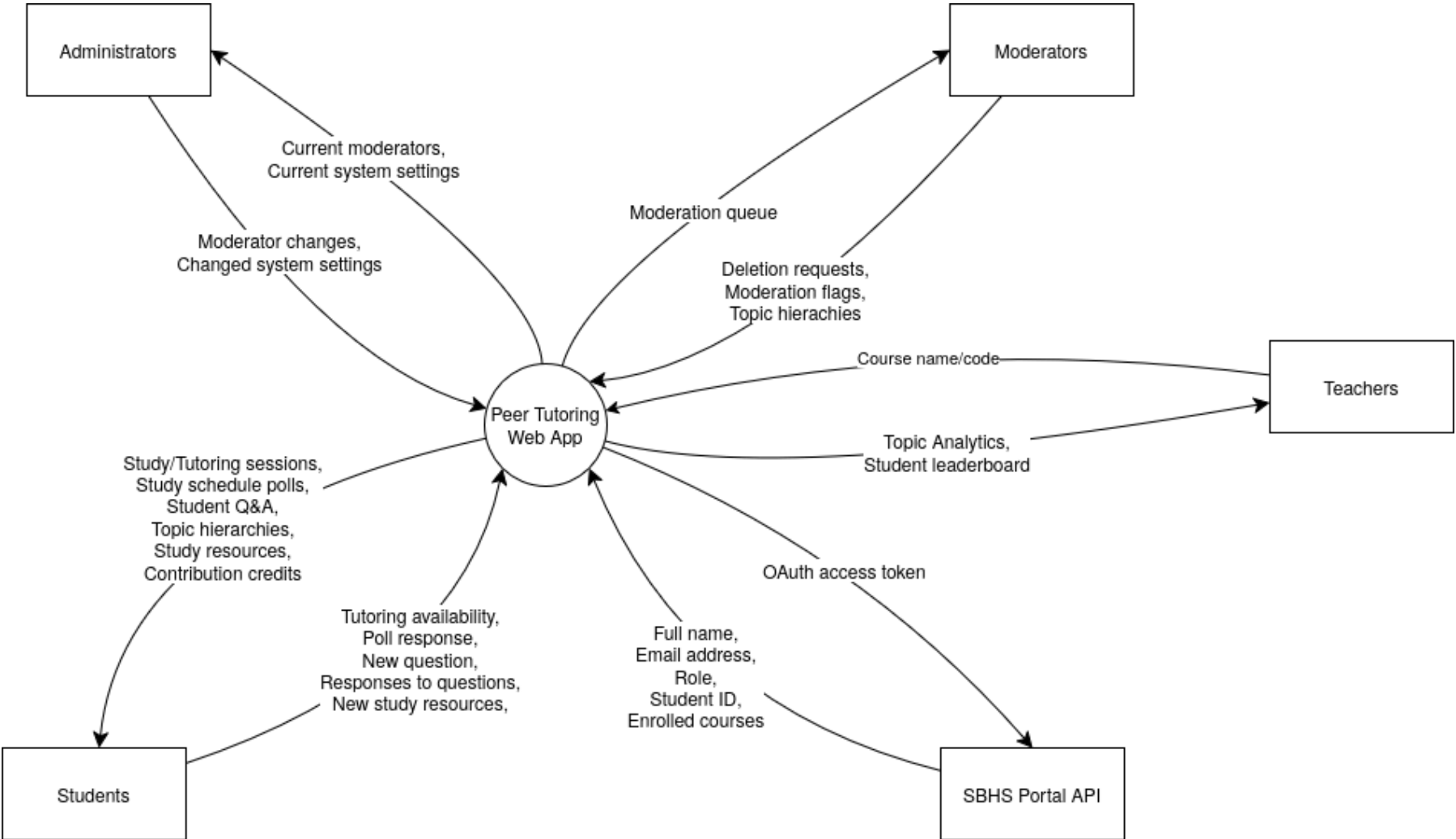
2.1. Gantt Chart



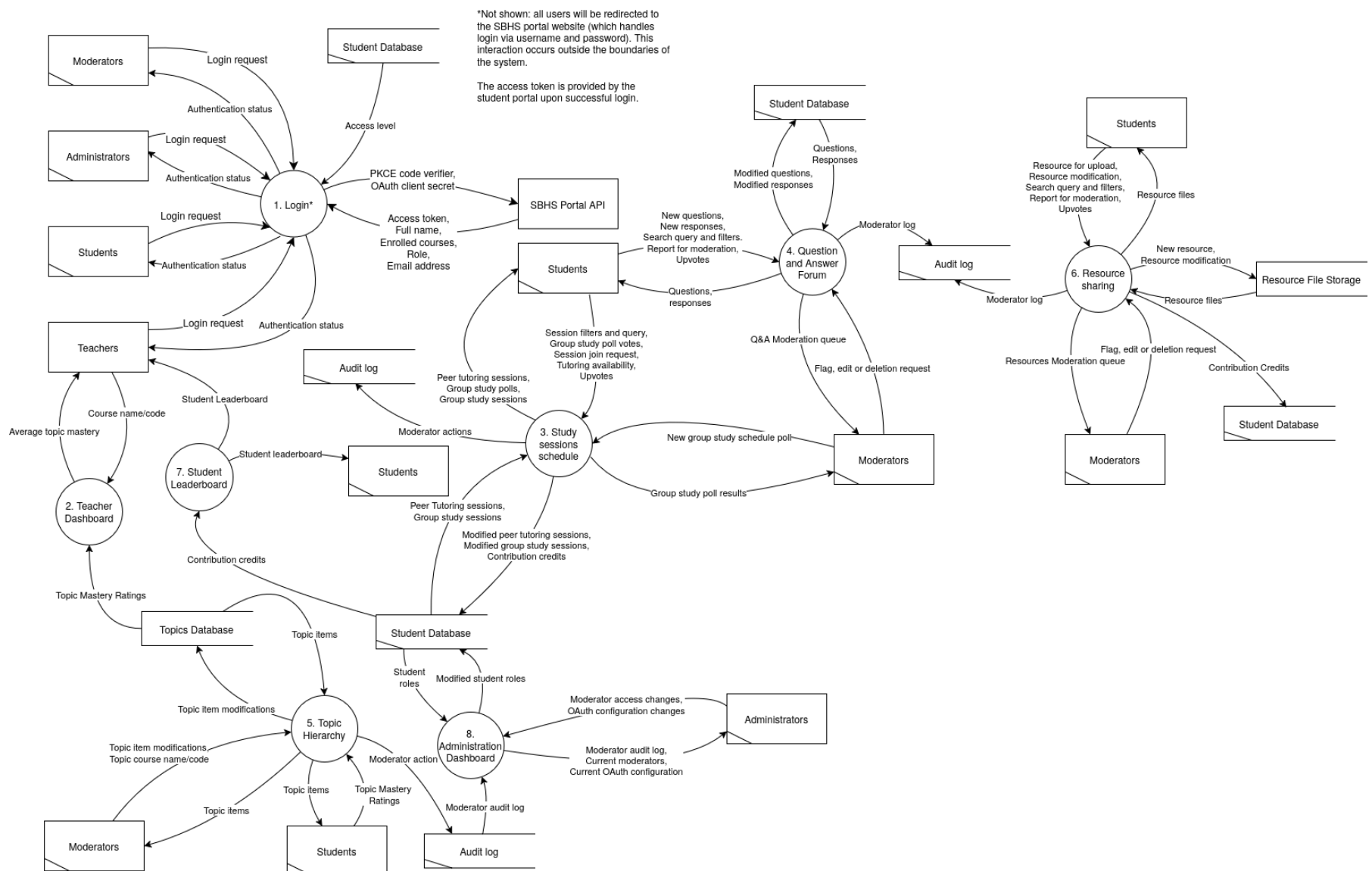
2.2. Design

2.2.1. Data flow Diagram

Context Diagram (Level 0)



Level 1

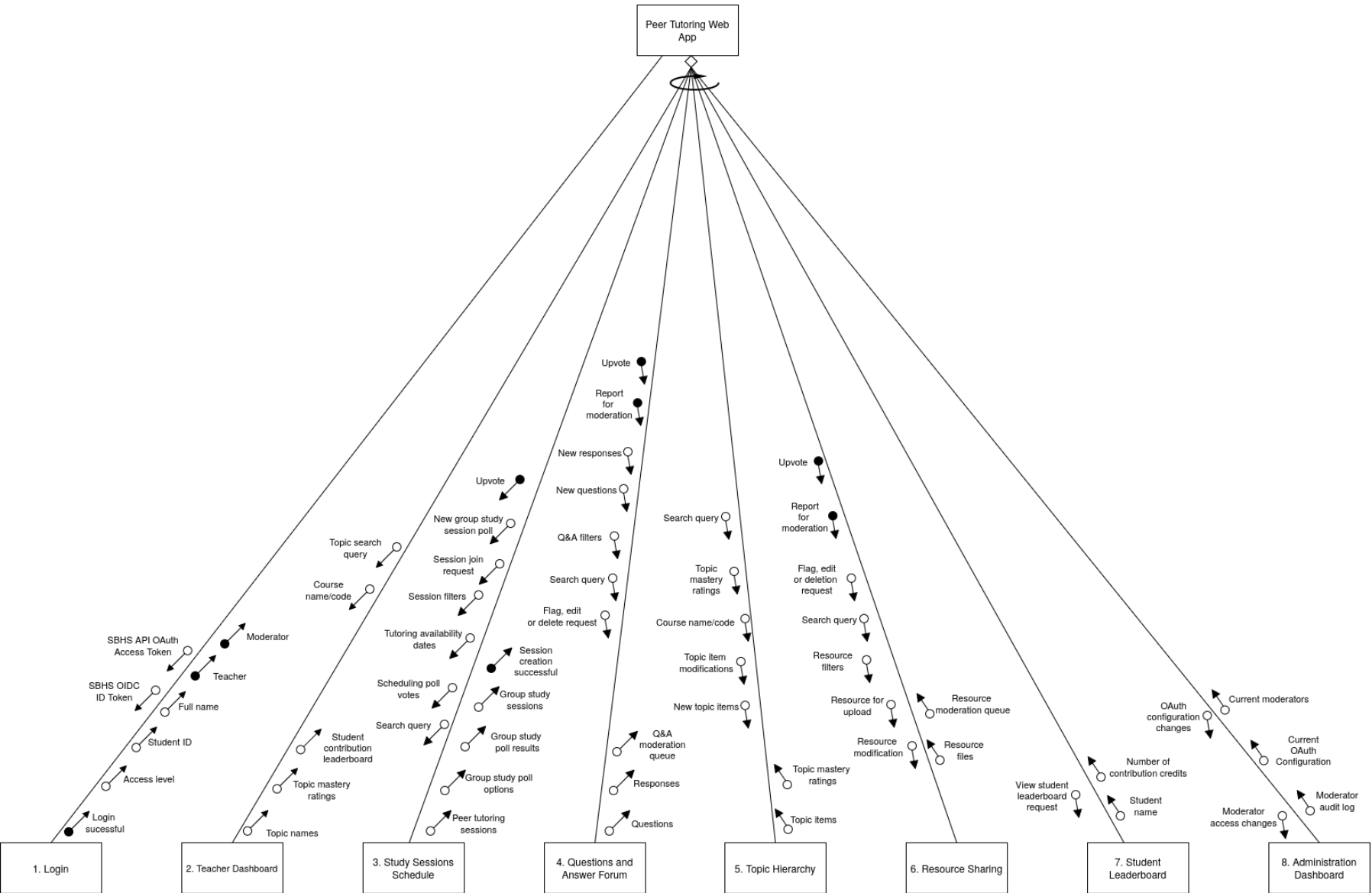


Level 2

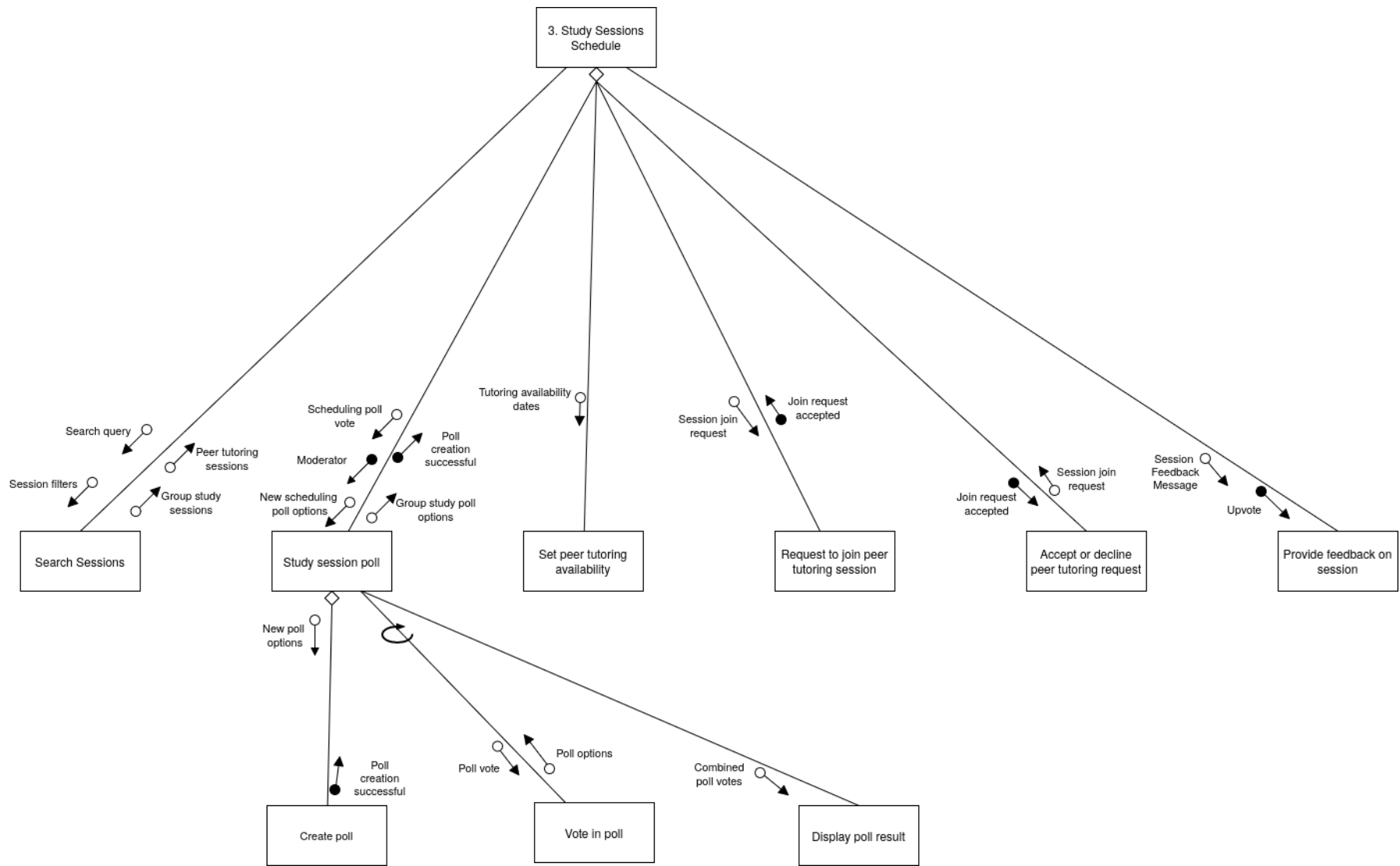


2.2.2. Structure Chart

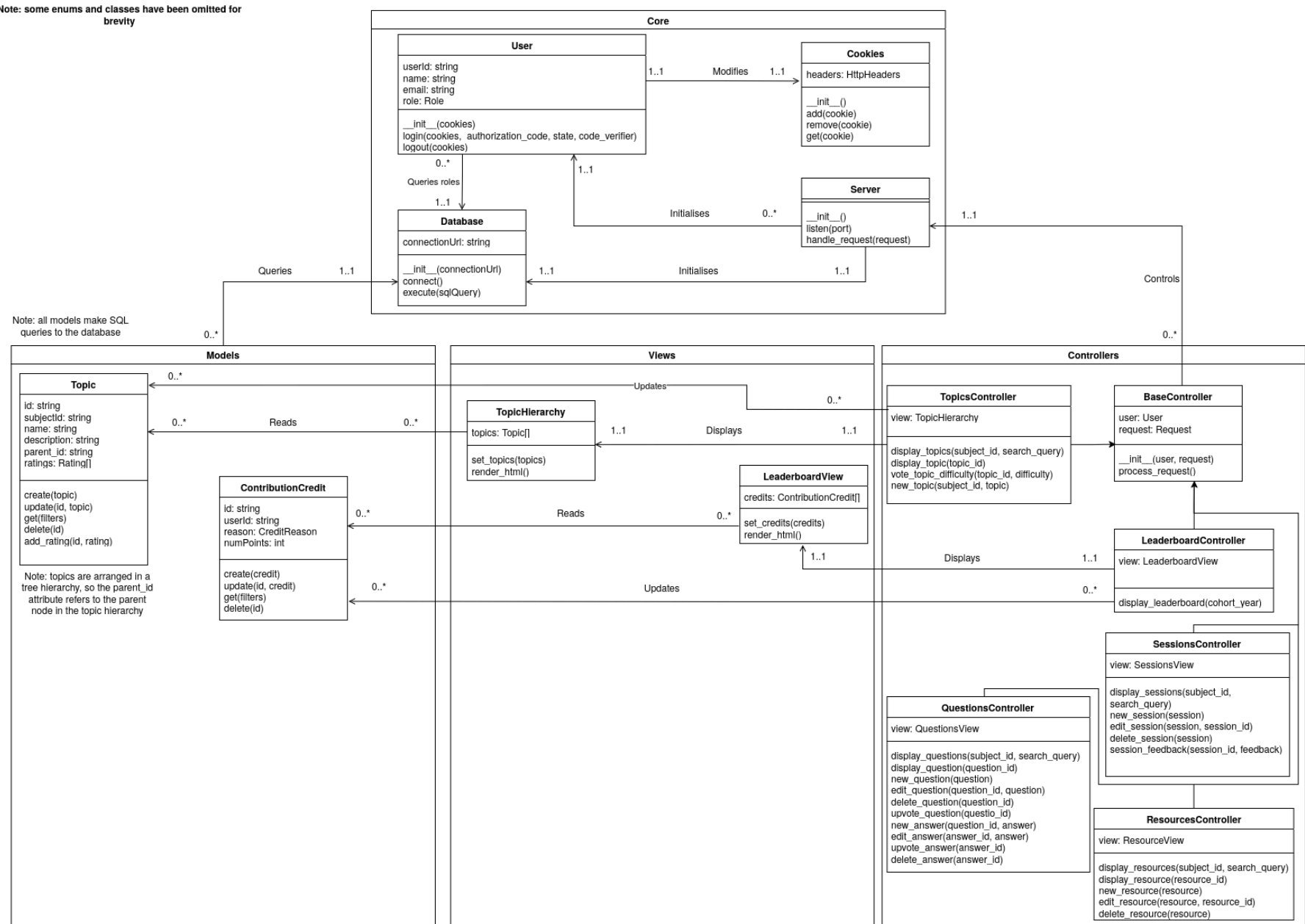
Level 1



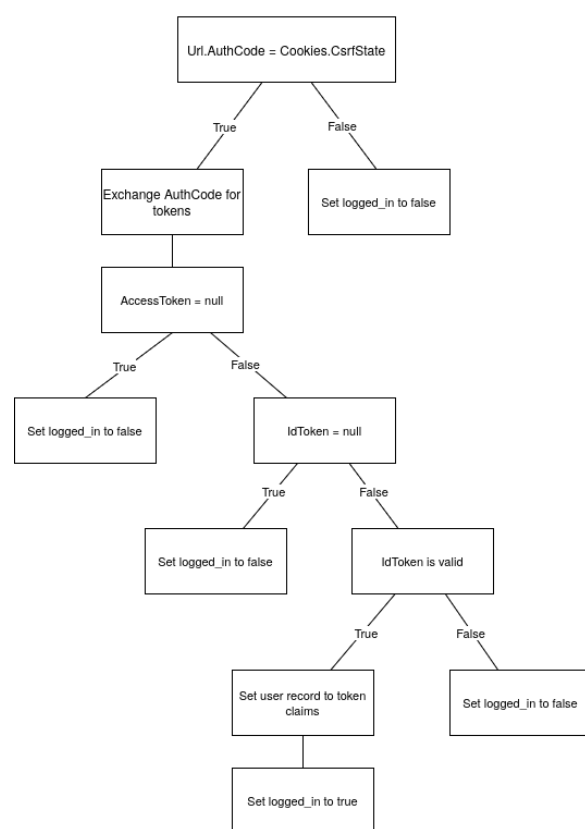
Level 2



Note: all models make SQL queries to the database



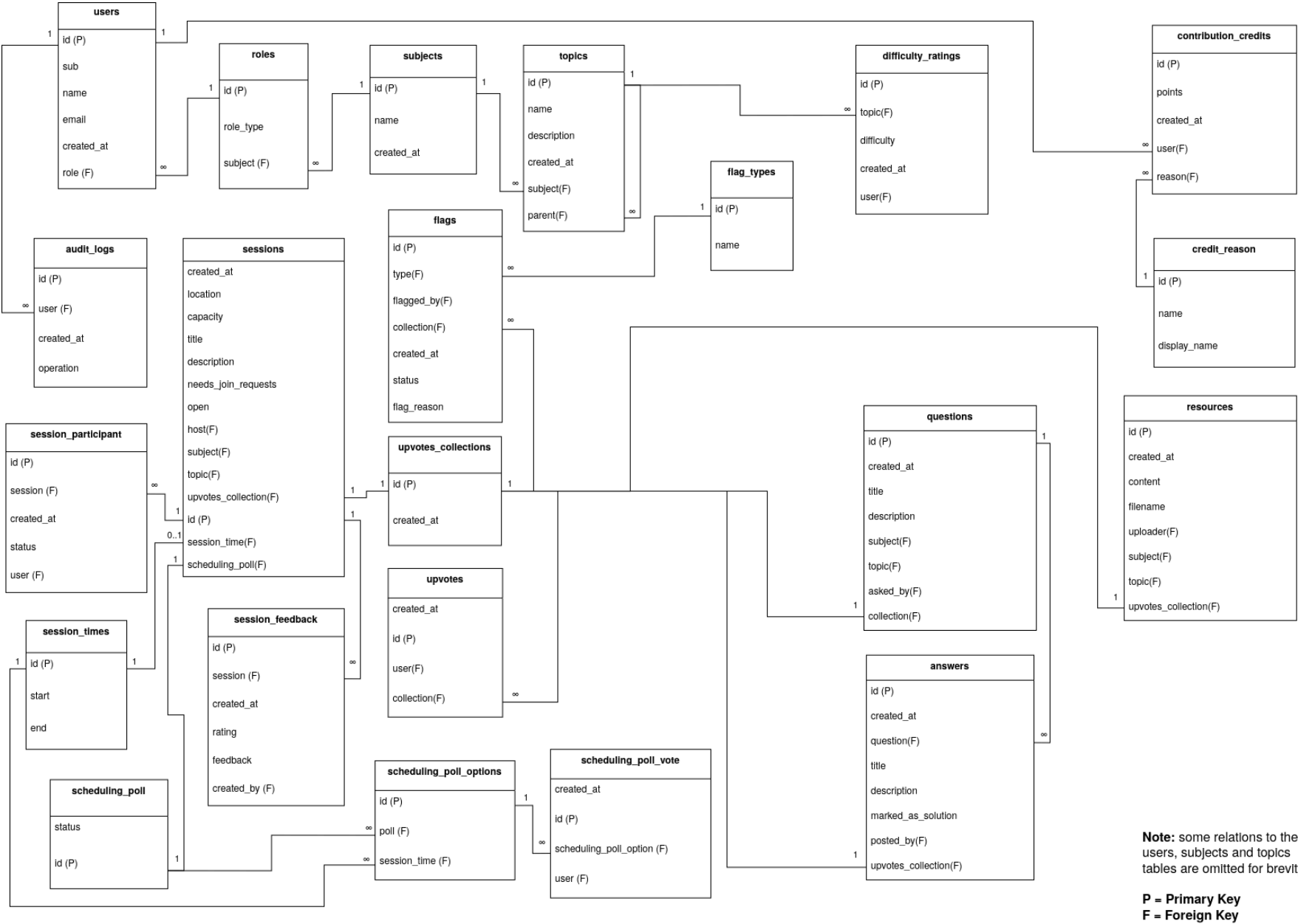
Login decision tree



```

graph TD
    A[Session.needs_join_requests] -- True --> B[Session.open]
    A -- False --> C[Search query = null]
    B -- False --> D[Current user is a session participant]
    B -- True --> E[Search query = null]
    D -- False --> F[Continue to next session]
    D -- True --> G[Search query = null]
    G -- False --> H[Session name or description matches search query]
    G -- True --> I[Display session]
    H -- False --> F
    H -- True --> J[Display Session]
    C -- False --> K[Session name or description matches search query]
    C -- True --> L[Display session]
    K -- False --> F
    K -- True --> M[Continue to next session]
    E -- False --> N[Session name or description matches search query]
    E -- True --> O[Display session]
    N -- False --> F
    N -- True --> P[Continue to next session]
  
```


2.2.5. Database Schema



2.2.6. Data Dictionary

Data Structure (type)	Attributes	Data type	Max length	Description
users (array of records)	id	integer	8 bytes	Primary key
	sub	string	100 chars	OIDC subject claim from school-portal supplied JWT
	name	string	70 chars	Full name of user (from school portal)
	email	string	254 chars	Email address of user (from school portal)
	created_at	timestamp	8 bytes	When the record was created
	role	integer	8 bytes	Role ID (foreign key to roles table)
	role_display	string	20 chars	Display name for role
	is_admin	boolean	1 byte	Whether the user has admin rights

Data Structure (type)	Attributes	Data type	Max length	Description
subjects (array of records)	id	integer	8 bytes	Primary key
	name	string	40 chars	
	created_at	timestamp	8 bytes	When the record was created
	topics	array of topic records	EOF	Subtopics of this subject

Data Structure (type)	Attributes	Data type	Max length	Description
topic (record)	id	integer	8 bytes	Primary key
	name	string	40 chars	
	description	string	EOF	
	created_at	timestamp	8 bytes	When the record was created
	subtopics	array of records	EOF	Subtopics of this topic

Data Structure (type)	Attributes	Data type	Max length	Description
difficulty_ratings (array of records)	id	integer	8 bytes	Primary key
	topic_id	integer	8 bytes	Foreign Key to topic
	created_at	timestamp	8 bytes	When the record was created
	user_id	integer	8 bytes	User who rated topic (foreign key to users)

Data Structure (type)	Attributes	Data type	Max length	Description
contribution_credits (array of records)	id	integer	8 bytes	Primary key
	points	integer	4 bytes	Number of points this contribution is worth
	created_at	timestamp	8 bytes	When the record was created
	user_id	integer	8 bytes	Recipient of the contribution credit (foreign key to users)
	reason_id	integer	8 bytes	ID of the credit reason (foreign key to credit_reasons)
	reason_name	string	20 chars	Internal name of credit reason
	reason_display_name	string	20 chars	Display name of credit reason

Data Structure (type)	Attributes	Data type	Max length	Description
audit_logs (array of records)	id	integer	8 bytes	Primary key
	user_id	integer	8 bytes	Moderator who performed the action (foreign key to users)
	created_at	integer	8 bytes	When the record was created
	operation	string	EOF	JSON description of the operation

Data Structure (type)	Attributes	Data type	Max length	Description
-----------------------	------------	-----------	------------	-------------

sessions (array of records)	created_at	integer	8 bytes	When the record was created
	location	string	100 chars	Voice channel or meeting link to join the session
	capacity	integer	4 bytes	Max number of students allowed to join the session
	title	string	30 chars	Title of the session
	description	string	EOF	Description of the session
	needs_join_requests	boolean	1 byte	Whether the session needs a request to join
	open	boolean	1 byte	Whether the session is open to new participants
	host	integer	8 bytes	User who hosts the session (foreign key to users)
	subject_id	integer	8 bytes	The subject the session is associated with (foreign key to subjects)
	topic_id	array of integer	EOF	Topics the session is associated with (foreign key to topics)
	id	integer	8 bytes	Primary key
	session_start	integer	8 bytes	Timestamp the session starts
	session_end	integer	8 bytes	Timestamp the session ends
	upvotes_id	integer	8 bytes	Foreign key to upvotes_collection
	scheduling_poll_id	integer	8 bytes	Foreign key to scheduling_poll
	participants	array of records	EOF	Participants in this session
	feedback	array of records	EOF	Feedback ratings given by session members

Data Structure (type)	Attributes	Data type	Max length	Description
participant (record)	id	integer	8 bytes	Primary key
	created_at	integer	8 bytes	When the record was created
	status_id	integer	8 bytes	The status of the students' request to join the session (foreign key to session_participant_statuses)
	status_display	string	20 chars	The display name for the status of the students' request to join the session
	user_id	integer	8 bytes	The participant (foreign key to users)

Data Structure (type)	Attributes	Data type	Max length	Description
feedback (record)	id	integer	8 bytes	Primary key
	created_at	integer	8 bytes	When the record was created
	rating	integer	4 bytes	Rating out of 5 for the session
	feedback	string	EOF	Optional written feedback for the session
	created_by	integer	8 bytes	Who created this feedback record (foreign key to user)

Notes:

- Character lengths are provisional lengths for the size of a single line when displayed.
- In SQLite, the max length of an INTEGER is 8 bytes, so that length is used for all IDs
- 8 byte Unix epoch timestamps are used, as 4 byte epoch timestamps cannot represent timestamps past 19 January 2038

2.2.7. Algorithms**Main program**

```
logged_in = false
User = Null
```

```
BEGINMAIN
  WHILE logged_in = false
```

Cohortly - Software Engineering Major Project

```
User = Login()
ENDWHILE

WHILE logged_in
  Get user input
  CASEWHERE user input
    'admin'      : AdminPage()
    'leaderboard' : LeaderboardPage()
    'sessions'   : SessionsPage()
    'questions'  : QuestionsPage()
    'resources'  : ResourcesPage()
    'topics'     : TopicsPage()
    'log out'    : logged_in = false
  END CASE
ENDWHILE
ENDMAIN
```

Login

```
BEGIN Login()
  CsrfToken = GenerateRandomString()

  RedirectToSbhsPortal()

  AuthCode = GetParamFromUrl('code')
  CsrfState = GetParamFromUrl('state')

  IF AuthCode <> CsrfState THEN
    logged_in = false
    RETURN Null
  ENDIF

  Response = ExchangeAuthCodeForTokens()

  AccessToken = GetFromResponse(Response, 'access_token')
  RefreshToken = GetFromResponse(Response, 'refresh_token')
  IdToken = GetFromResponse(Response, 'id_token')

  IF AccessToken = Null THEN
    logged_in = false
    RETURN Null
  ENDIF

  IF IdToken <> Null THEN
    TokenValid = VerifyToken(IdToken)      'Note that this uses digital signature algorithms (DSA) to verify the token
rather than a database search
    IF TokenValid THEN
      Claims = GetUserClaims(IdToken)
      logged_in = true
      RETURN CreateUserRecord(Claims.id, Claims.name, Claims.email)
    ENDIF
  ENDIF

  RETURN Null
END Login
```

Leaderboard

```
BEGIN LeaderboardPage()
  CohortYear = Get user input

  LeaderboardRecords = []

  Open StudentDatabase
  Open CreditsDatabase

  FOR i = 0 to StudentDatabase.Length
    StudentRecord = StudentDatabase[i]

    'This is implemented using an SQL WHERE clause
    CreditRecords = FilterByUserId(CreditsDatabase, StudentRecord.UserId)

    Points = 0
    For j = 0 to CreditRecords.Length
      Points = Points + CreditRecords[j].Points
    NEXT j

    LeaderboardRecord = Empty leaderboard record
    LeaderboardRecord.Student = StudentRecord
    LeaderboardRecord.Points = Points
  NEXT i

  MergeSortDescendingByPoints(LeaderboardRecord)

  Close CreditsDatabase
  Close StudentDatabase

  FOR i = 0 to 29
    IF i < LeaderboardRecords.Length THEN
      Record = LeaderboardRecords[i]
      Display 'Place', i + 1
      Display 'Name', Record.Student.Name
      Display 'Points', Points
    ENDIF
  NEXT i
END LeaderboardPage

BEGIN MergeSortDescendingByPoints(Records)
  IF Records.length <= 1 THEN
    RETURN Records
  ENDIF

  Middle = Records.length / 2

  Left = MergeSortDescendingByPoints(Records from index 0 to Middle)
  Right = MergeSortDescendingByPoints(Records from index Middle + 1 to Records.length - 1)

  RETURN MergeByPointsDescending(Left, Right)
END MergeSortByPoints

BEGIN MergeByPointsDescending(Left, Right)
  MergedArray = []
  WHILE Left is not empty AND RIGHT is not empty
    IF Left.Length > 0 AND Right.Length > 0 THEN
      IF Left[0].Points > Right[0].Points THEN
        Append Left[0] to MergedArray
        Remove Left[0]
      ELSE
        Append Right[0] to MergedArray
        Remove Right[0]
      ENDIF
    ELSE IF Left.Length > 0 THEN
      Append Left[0] to MergedArray
      Remove Left[0]
    ELSE IF Right.Length > 0 THEN
      Append Right[0] to MergedArray
      Remove Right[0]
    ENDIF
  ENDWHILE

  RETURN MergedArray
END MergeByPointsDescending
```

Questions Page

```
BEGIN QuestionsPage()
  Get user input
  CASEWHERE user input
    'search_questions' : QuestionsSearch()
    'new_question'     : NewQuestion()
    'edit_question'    : EditQuestion()
    'delete_question'  : DeleteQuestion()
    'upvote_question'  : UpvoteQuestion()
    'new_answer'       : NewAnswer()
    'edit_answer'      : EditAnswer()
    'delete_answer'    : DeleteAnswer()
    'upvote_answer'    : UpvoteAnswer()
  END CASE
END QuestionsPage
```

Session Page

```
BEGIN SessionsPage()
  Get user input

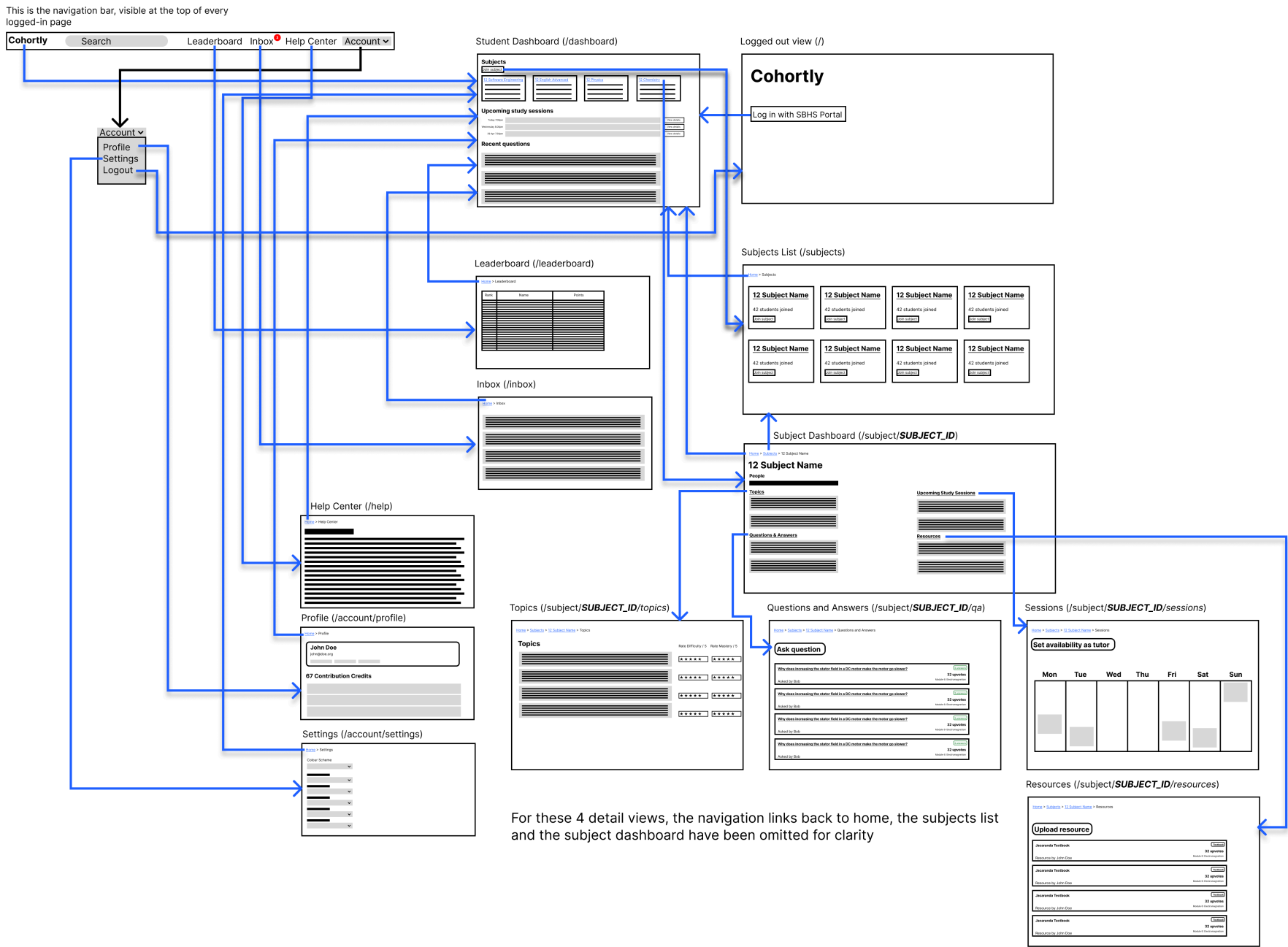
  CASEWHERE user input
    'search'           : SessionsSearch()
    'group_polls'      : SessionsPoll()
    'peer_tutoring'    : SessionsPeerTutoring()
    'feedback'         : SessionsFeedback()
  END CASE
END SessionsPage

BEGIN SessionsSearch()
  Get user input
  Open SessionsDatabase

  Index = 0
  WHILE Index < SessionsDatabase.Length
    IF Contains(SessionsDatabase[Index].name, user input) OR
       Contains(SessionsDatabase[Index].description, user input) THEN
      'Note that the implementation will use a fuzzy
search algorithm for more relevant search results
      Display 'Name', SessionsDatabase[Index].name
      Display 'Description', SessionsDatabase[Index].description
      Display 'Subject', SessionsDatabase[Index].subject
      Display 'Date', SessionsDatabase[Index].date
      Display 'Time', SessionsDatabase[Index].time
      Display 'Location', SessionsDatabase[Index].location
      Display 'Host', SessionsDatabase[Index].host
    ENDIF
    Index = Index + 1
  ENDWHILE

  Close SessionsDatabase
END SessionsSearch
```


2.2.8. Storyboard



Zoom for detail.

2.2.9. Interface Design

Consistency

Responsive Web Design

Navigation

Grouping

Effective use of colour

Accessibility

3. Producing and Implementing

3.1. Logbook

3.1.1. 16 November 2025

At this stage I have some of my early specifications, so I can start considering what technologies to use for implementation:

Since my project is collaborative, functionality has a heavy dependency on the shared database. Most of the content in the web app is located on the server, so there is not much point having a JS-first application “shell” syncing server side state, since the shell can’t do much on its own. Thus a server-first implementation, with content rendered to HTML on the server makes the most sense. Additionally, offline functionality can still be available through read-only service worker caching of rendered pages.

Technology stack: Rust-based server side libraries: Axum for server Maude for templating (html generated by rust macros) This allows for a JSX-like component system except its entirely server-side Lightweight client side libraries: Alpine.js for simple client side interactivity Possibly solid or svelte for any highly interactive component islands (but loaded only in those places) HTMX for no-reload form and link UX Pico css for semantic html styling

Authentication: Server-side OAuth: students login with student portal, their ID token, access token, refresh token are all stored in HTTP-only cookie

3.1.2. 7 January 2026

Completed authentication middleware: Can now log in with the SBHS portal, interfacing with OAuth and OIDC The middleware extracts name, user ID and email address from the ID Token (JWT)

3.1.3. 21 April 2026

Completed database integration. I chose SQLite for the database for its ease of deployment. SQLite is a bit more bare in functionality compared to some heavier databases and suffers from relaxed typing, but using it means that the system can be deployed as a single container with a persistent volume attached. If this ends up being student-run that is very useful. Can now create courses in the database which students can join I chose breadcrumb navigation to have a consistent way for users to navigate the hierarchical interface.

3.1.4. 12 July 2026

Finished implementation of subjects and topics in the peer tutoring system.

3.1.5. 18 July 2026

Began rewrite of major project to Python with Django. The axum-based stack proved unproductive since the project required many repetitive forms. This proved far more efficient in Django, compared to axum where the form HTML must be written manually.

Most of the HTML and styles could be carried over without too much change, so much of the work went into backend improvement. Despite a slight initial learning curve, after very few hours this approach proved very productive.

3.1.6. 19 July 2026

Completed implementation of all existing features in the new Django-based system. Including:

- Authentication
- Subjects
- Initial topics system

3.1.7. 21 August 2026

Finished topic system, allowing moderators to create, edit and reorder topics. Topic descriptions are markdown text, allowing for features such as headings, code and images to be included in the description. This markdown system was planned to be reused in other areas of the web application, allowing for rich text in questions and answers.

3.1.8. 23 August 2026

Completed question and answer system, including upvoting questions and answers, and marking an answer as a solution. Also created a search feature, re-implemented profile and settings menus, and much of the initial study sessions scaffolding.

3.1.9. 24 August 2026

Implemented interactive calendars using the `fullcalendar.js` library, supporting efficient visualisation of study sessions. A similar calendar was also included grade-wide to allow the prevention of scheduling conflicts between different subjects.

3.1.10. 30 August 2026

Implemented resources, peer-tutoring sessions, virus scanning (and email reminders) and email alerts for uploaded viruses. Installed onto live system at <https://cohortly.up.railway.app/>

3.2. User documentation

This is the user documentation included in the product.

3.2.1. Getting Started

Cohortly is a web application designed to foster student collaboration by allowing students to share their knowledge in Q&As and study sessions.

To use Cohortly, you log in with your School Portal account. Your account is linked to your email address, student ID and full name. Access is limited to Sydney Boys High School students only. Once logged in, you will be redirected to the Dashboard

3.2.2. Joining a subject

From the dashboard, you can join a subject by clicking the "Join or leave subjects" button. From there you can manage your subject memberships. Joining a subject is required before you can use the study sessions or Q&A features. From there, you can click on a subject to view its detail page.

3.2.3. Asking and answering questions

From the subject page, you can click on the "Questions" link in the sidebar to access the questions list. From there, you can ask a question, or view existing questions. Within a question, you can upvote the question if you find it helpful, post an answer, and upvote answers. Upvoted questions and answers appear earlier in their lists, allowing people to find them more easily. If you posted a questions, you can mark the answer that best answers it as the solution to your question, where it will appear at the top of the answers list to help people find it more easily. Since questions serve as a resource for other students, they should be clear and well-written, asking a specific syllabus-relevant question.

3.2.4. Uploading and using resources

You can upload resources (past papers, study notes, textbooks) as either a PDF document, or as a link to Google Docs or some other website. Uploaded documents must be in PDF format and be under 10 MB in size. Documents will be scanned for viruses before being visible to other students.

3.2.5. Joining study sessions

Group study sessions are created by moderators and are designed to be larger general review sessions to assist many students in reviewing and catching up similar content. Peer tutoring sessions can be created by anyone, and allow a small group or one-on-one study sessions to catch particular students up on content they may have missed or be weaker at. On your dashboard, you can see the study sessions for all subjects across the Cohortly instance for your grade. Clicking on any entry will send you to the session detail page for that session, showing description, time, location, participants and other relevant information. If you are a member of

that subject, you can join the session by clicking on “Join session” on the session detail page. Sessions filtered to a subject can be accessed by clicking on “Sessions” in the sidebar on any subject page. **Participating in peer tutoring** You can create a peer tutoring session by clicking “Create a peer tutoring session” on the subject’s session page. This creates a new empty peer tutoring session, which will appear to other students as long as there are spots available. You can select a capacity up to 8 (but smaller groups of 2-3 are recommended) for that session, after which it will be automatically closed to new participants. You can manually set the open or closed status of the session by clicking on the “Edit session” button and ticking or unticking the “open” checkbox. When students join your session, you can choose whether to accept or decline their join request.

3.2.6. Frequently Asked Questions

Why can’t I join a session?

Sessions are capacity limited to a number of students decided by the host. This prevents groups from becoming too large and making it harder to support students. Session participation is assigned on a first-come-first-serve basis. When a session is at capacity, the join button will be grayed out.

Why can’t I see my subject on the dashboard?

Your dashboard will only show a subject you are a member of. If you have not yet joined a subject it will not be visible on that page. If your subject is not visible on the subjects list page, then you will need to contact the administrator to add the subject.

How are new subject requests processed?

Before a new subject is created, a moderator who will manage group study sessions and Q&A content needs to be appointed. More than one moderator can be appointed for large subjects to split the workload. Moderator choice is subject to the administrator’s discretion, and moderator privileges can be revoked if abused. A moderator can only moderate content for their appointed subject.

How does moderation work?

Moderators chosen for each subject by an administrator can create and modify group study sessions. They can also modify or delete questions and answers if they contain inappropriate content.

4. Testing and Evaluating

4.1. Test Report

4.1.1. Authentication

This is handled by the student portal.

Input (username)	Input (password)	Expected output	Reason
4440000000	Correct corresponding password	The main dashboard is shown	Testing successful authentication
bob	banana	Error message: “Invalid username or password”	Testing non-existent accounts
4440000000	banana	Error message: “Invalid username or password”	Testing incorrect password

4.1.2. Sessions

Create session

Input (session type)	Input (start date and time)	Input (end date and time)	Input (Capacity)	Expected output	Reason
Peer tutoring	30/08/2026 5:00pm	30/08/2026 5:30pm	2	Error message: “Date must not be in the past”	Testing cannot make past sessions
Peer tutoring	31/08/2026 5:00pm	31/08/2026 5:00pm	2	Error message: “End time must be after start time”	Testing cannot make zero-time session
Peer tutoring	31/08/2026 5:00pm	31/08/2026 4:00pm	2	Error message: “End time must be after start time”	Testing cannot make end before start
Peer tutoring	31/08/2026 5:00pm	31/08/2026 5:40pm	8	Peer tutoring session is shown	Testing can make peer tutoring up to 8 people
Peer tutoring	31/08/2026 5:00pm	31/08/2026 5:40pm	0	Error message: “Capacity must be greater than 0”	Testing capacity is a positive integer
Peer tutoring	31/08/2026 5:00pm	31/08/2026 5:40pm	-1	Error message: “Capacity must be greater than 0”	Testing capacity is a positive integer
Peer tutoring	31/08/2026 5:00pm	31/08/2026 5:40pm	9	Error message: “Capacity must be at most 8”	Testing peer tutoring capacity limit

Group study session	31/08/2026 5:00pm	31/08/2026 5:40pm	250	Group study session shown	Testing group study capacity limit
Group study session	31/08/2026 5:00pm	31/08/2026 5:90pm	250	Error message: "Invalid time"	Testing invalid time
Group study session	32/08/2026 5:00pm	32/08/2026 5:30pm	250	Error message: "Invalid date"	Testing invalid date
Group study session	31/08/2026 5:00pm	31/08/2026 5:40pm	251	Error message: "Capacity must be at most 250"	Testing group study capacity limit

4.1.3. Resources

Create resource

Input (Resource title)	Input (Description)	Input (File upload)	Expected output	Reason
	Resource description	-	Error message: "Title cannot be blank"	Testing blank title
Study resource	Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.	-	Resource page is shown	Testing creation of resource with no file
Study resource		image.png	Error message: File must be a valid PDF	Testing upload of non-PDF
Study resource		image.pdf	Error message: File must be a valid PDF	Testing upload of corrupted PDF
Study resource		virus.pdf	Resource page with error message is shown: "This file has been flagged as a virus"	Testing virus scanning
Study resource		-	Error message: "Include either a file upload or description"	Testing blank resource

4.1.4. Questions page

Create question

Input (Question title)	Input (Question body)	Expected output	Reason
Test	Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.	Error message: "Title must be greater than or equal to 15 characters"	Testing short title
15CharactersLen	Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.	Question page is shown	Testing exactly 15 characters long title
How do I write good test data?		Error message: "Description must be greater than or equal to 30 characters"	Testing blank description
How do I write good test data?	Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore. 	Question page is shown with no image.	Testing sanitation of input data to thwart XSS attacks
How do I write good test data?	Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore. User drag and drops image	Question page is shown with image.	Testing drag and drop of image
How do I write good test data?	Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore. User drag and drops PDF	Question page is shown with no file.	Testing only images allowed on question page
How do I write good test data?	Lorem ipsum dolor sit amet, consectetur adipiscing elit,	Question page is shown with equation:	Testing formatting of mathematical equations

	sed do eiusmod tempor incididunt ut labore. \$ E = mc^2 \$	$E = mc^2$.	
How do I write good test data?	this is bolded text and <i>italics</i>	Question page is shown with proper formatting	Testing formatting of text in question (this is bolded text and <i>italics</i>)

4.1.5. Reccomendations

This system passes preliminary testing of the main cases, as well as some user inputs designed to test input santisation and resistance to XSS attacks. However, this testing methodology lacks more sophisticated testing such as load testing, automated fuzzing (DAST), and testing for race conditions. These tests should be perfromred before installation of the system.

4.2. Evaluation

1. The system should allow teachers and students to log in using the school portal system ✓ (Partial)

Students can log in but teacher support was dropped due to time limitations

The system should allow students to:

- Post Q&As ✓
- Schedule or join sessions ✓
- Share resources ✓
- Set peer tutoring availability ✓

All of these features are fully implemented

The system should allow moderators to:

- Remove inappropriate or irrelevant content ✓
- Review flags for out-of-syllabus or disputed material ☒
- Add subtopics to subjects ✓

Essential moderation features such as removing and editing content as well as creating and modifying subtopics was implemented, but more advanced moderation features were omitted due to time limitations.

The system should allow system administrators to:

- Add or remove moderators ✓
- Create and manage subjects ✓

This was fully implemented (admin dashboard).

The system should allow teachers only to:

- View statistics on platform use (contribution credits) and student topic mastery ratings ☒

Teacher access was not implemented.

Scheduling and Study Sessions

Students should be able to view public study sessions/tutoring sessions, and filter by: ✓ (Partial)

- Subject
- Host (tutor or subject moderator)
- Date/time
- Sessions should show:
- Subject or topic
- Session title and description
- Date and time
- Meeting link or location
- Host (student or moderator)
- Participant list
- Maximum capacity

All of these features are implemented and are able to be viewed and searched by, but advanced filtering by these properties was not implemented.

Students should be able to:

- Set availability as a peer tutor: ✓
- Set availability for tutoring sessions ✓
- Set size (max number of students) of session ✓
- Accept or decline requests ✓

This was fully implemented.

Subject moderators should be able to create general study sessions: ✓

- General study sessions timing is decided by scheduling polls ☒

Students should be able to vote in scheduling polls: ☒

- Students vote for their available times, and the most popular time slot is chosen ☒

While general study sessions were implemented, the polling feature was omitted due to time limitations.

Topic Hierarchy

The system should represent topics in a hierarchical way: ✓

- Topics are associated with a subject, and topics can have sub-topics corresponding to increasing detail

Students should be able to vote on the difficulty and mastery (how much they understand) of a topic ☒ Teachers should be able to view aggregated statistics on difficulty and mastery metrics ☒

While the topic hierarchy can be created and modified by moderators, difficulty rating was not implemented.

Moderation

Students should be able to flag content as: ☒

- Inappropriate
- Out of syllabus
- Disputed

Moderators should be able to see a queue of flagged content, and choose an action (accept flag/delete/reject flag) ☒ The system should store all moderator actions in an audit log, so moderator actions are held accountable to the moderator ☒

Moderators can manually edit or remove harmful content but these advanced moderation features were not implemented. At the scale of a student cohort where students' names are associated with their actions, the moderation problem is far more tractable so these were not as important.

Q&A

Students should be able to: ✓

- Post questions, associated with a topic in the topic hierarchy
- Respond with answers
- Flag a question

These were fully implemented

Resource Sharing

1. To share notes, worksheets and study materials, students should be able to: ✓
 - Share links
 - Upload files
2. The system should tag resources with: ✓
 - Subject
 - Topic
 - Uploader
 - Upload date
3. The system must scan uploaded files with VirusTotal API or similar ✓
4. The system must limit to safe file formats (PDF, DOCX, PPTX, etc.) ✓ (Partial)
5. Students should be able to flag resources ✓

This was fully implemented except resources can only either be PDF or link to external content (eg Google Docs), for ease of validation.

Points system

1. Students should be able to upvote resources, Q&A, and study sessions to indicate that they were helpful
2. After a study sessions, participants should be able to provide anonymous feedback on a study session, including:
 - A rating out of 5
 - General feedback
3. The system should provide students with **contribution credits** for participating, with higher credit amounts for more helpful activities
4. The system should display a leaderboard for the students with the highest credit amounts

4.2.1. Non-Functional

Usability

1. The system must be functional on desktop and mobile devices
2. The most commonly used actions must be easy to access intuitively
3. The interface must be consistent
4. There must be a help page to explain more complex processes (such as moderation, the contribution credits system, etc.)

Security and Privacy

1. Only students or teachers logged in through the school portal should be able to view any data within the system
2. Uploaded content should be automatically scanned with VirusTotal and restricted to safe formats